

Academic Statement of Purpose

Avery Reyna — averyreyna.github.io — University of Michigan Information

Motivation. Software construction should not be reserved solely for those with high technical aptitude; it should be available to everyone, regardless of academic background. As natural language prompting becomes more ubiquitous in programming, these interfaces must bridge the gap between intent and implementation. **By working at the intersection of AI, human-computer interaction, and software engineering, I aim to make coding more approachable and intuitive for non-technical users.** My goal is to expand who can build software in the first place, ensuring the tools shaping our world reflect diverse perspectives, not just those with traditional CS backgrounds—a barrier I’ve navigated myself. I bring to this work two years of industry experience building developer tools at Capital One, a self-taught programming background, and firsthand research leadership, currently co-directing a mixed-methods study at UC San Diego that examines the agentic coding workflows of experienced developers to reveal how these tools can be used more effectively by all types of engineers.

Research Experience. As an interdisciplinary social science student at the University of Central Florida, I took a wide breadth of courses that exposed me to various fields of study that informed my research interests. While taking classes in applied sociology, I worked under **Amy Donley** to create maps that illustrated overdose hotspots across Orlando, serving as evidence for white papers advocating for improved local drug policy. Before this project, I hadn’t realized that GIS software isn’t merely an aid for developing interactive maps—it’s what makes spatial patterns visible in the first place, effectively blocking certain research questions behind technical expertise. Indeed, this realization sparked my interest in developing my technical skills and raised broader questions about how technical ability determines the boundaries of inquiry across research domains.

To develop these technical skills, I took introductory CS classes and, after hours, studied Python and React to build web apps and steadily improve as a programmer. I applied my learnings to join **Pamela Wisniewski’s STIR Lab**, aiming to improve online risk detection for adolescents by building more ecologically valid datasets for training ML models. Similar to my previous experience, I was fascinated by the role of software in our research, most of which was developed in-house. I qualitatively coded 50,000 Instagram direct messages using custom-built annotation software tailored to our codebook, streamlining insight identification far more than any off-the-shelf alternative ever could. Working with a dataset of

this scale inspired me to independently lead an NLP project to test whether text summarization models could streamline qualitative analysis for large datasets like ours. These experiences revealed that programming skills fundamentally change what's possible for users: rather than constraining research to fit existing software, you can build tools that fit your specific use case, making me wonder how many researchers remain limited by their inability to do the same.

Ending my journey in computational social science with **Jonathan Powell** to maintain *CoupCast*, an ML system used to predict global illegal leadership turnover, I began interrogating the implicit dependency between domain experts and programmers. Building data pipelines and interactive visualizations—work that supported public scholarship I co-authored for *Political Violence at a Glance*, *Just Security*, and *The Loop*, reaching over half a million readers—required software development skills that traditional social science training rarely includes. When I worked with tools like *Datawrapper*, I realized that accessibility didn't have to mean limited capabilities; complex customization and sophisticated logic could co-exist with user-friendly design. It raised a question that has been a north star for my graduate school goals: could we apply these design principles to coding and software construction more broadly, allowing domain experts to build their own custom tools rather than relying on professional programmers to actualize their vision?

Research Opportunities. Over time, I observed the rapid development of LLM-based programming tools, recognizing that they offered something fundamentally different from previous attempts at accessible coding interfaces. Unlike traditional tools that abstract code entirely through GUIs, LLMs could transform non-programmers into programmers by scaffolding direct engagement with code, helping them write, understand, and manipulate it themselves rather than avoiding it altogether. These weren't theoretical possibilities either, as being a Software Engineer in enterprise, I observed how LLM-assisted programming was already transforming who could build software. People without traditional programming backgrounds were now constructing small applications to explore solutions, accomplishing what had previously required an engineer for each step. This confirmed what I had suspected: with the proper scaffolding, programming could become accessible without compromising on power, customization, or the learning that comes from directly engaging with code.

Research Interests. Having witnessed both the need for accessible programming among domain experts and early evidence of LLMs' potential to address this gap, I want to investigate how we can design

AI-assisted programming tools that truly democratize software construction without sacrificing agency. I want to study how non-CS domain experts actually interact with AI coding assistants: the mental models they bring, where they struggle, and the practices that emerge as they transition from non-programmers to builders. Alongside this, I want to explore how to design interfaces that scaffold programming without abstracting it away, creating experiences that guide users through code construction while maintaining their direct engagement with it. Additionally, I'm interested in understanding what pedagogical approaches work when AI is part of the learning process—how to balance code generation with explanation, constrain AI capabilities to encourage learning, and help users develop programming knowledge.

Aligning Interests. I hope to collaborate with **Steve Oney** and **Eytan Adar**, whose work collectively seeks answers to questions driving my own research. Oney's focus on enabling and encouraging more people to write code with new programming tools is precisely what I want to dedicate my career to. His programming-by-demonstration systems like *ScrapeViz* and *ParamMacros* demonstrate how to better the process of constructing software while maintaining user retention of high-level concepts, a balance I aim to explore with LLM-assisted tools. My prior research experience with blind and low-vision users informs my interest in Oney's accessibility work, particularly his research on programmers with visual impairments and inclusive code readability—which could naturally extend to studying how AI coding assistants can better serve users with disabilities. These alignments suggest numerous potential projects that both advance the field and mirror Oney's initial work, including: investigating how AI coding assistants better enable the creation of software through novel interactions, exploring audio interfaces for code construction now that agentic coding workflows are gaining popularity, or more broadly examining how AI tools can make programming more accessible to diverse populations.

Adar's research at the boundaries of AI and HCI offers a complementary perspective essential to understanding how AI can democratize programming across disciplines. His work spans language, creativity, data mining, visualization, and social network analysis—reflecting the interdisciplinary breadth that defined my own research trajectory. This encapsulates how I want to study AI-assisted programming: not as an isolated technical problem, but as one that intersects with the diverse domains where non-technical experts need to build custom tools. His focus on making complex analytical capabilities accessible to non-experts directly parallels my interest in making programming accessible without sacrificing depth, particularly for users who bring rich domain knowledge but lack traditional CS training. Working with

Adar would allow me to investigate AI coding assistants within the varied disciplinary contexts where they're actually needed, examining how different communities of practice adopt and adapt these tools and ensuring that accessible programming serves the interdisciplinary research landscape.

Conclusion. Upon completing my PhD, I plan to continue researching and teaching students in academia. In addition to my standard responsibilities, I want to establish a program that invites community members, independent researchers, and industry professionals to join the lab and contribute to existing projects or lead their own studies that align with our interests. Many talented artists, designers, and engineers are committed to pursuing science in their free time, but are unable to do so because of the institutional barrier of not being affiliated with a university. Speaking from personal experience, I started *Self Adjoint Operator* to showcase my AI prototypes—work that opened doors to academic collaboration despite my lack of institutional affiliation and cemented my desire to research AI-assisted programming in graduate school. I want to be an outlet for people like me, making an explicit statement that collaborating with a diverse array of researchers can and will make my university's research presence that much more vibrant.